Genetic Algorithm for the 0/1 Knapsack Problem

A student project report on implementing a Genetic Algorithm first in Python (following a YouTube tutorial and the Whitley GA paper), then porting the same algorithm to MeTTa.

1. The task

My task was to implement a Genetic Algorithm in MeTTa that solves the Knapsack Problem. I read the Whitley GA tutorial paper that was given to me, and I watched a YouTube tutorial to understand the basics of how a GA works step by step.

The 0/1 Knapsack Problem is simple to describe. We have a set of items, each with a weight and a value. The knapsack has a maximum weight capacity. We need to pick which items to put in the knapsack so the total value is as large as possible without going over capacity. The 0/1 part means each item is either fully included or fully excluded. There is no taking half an item.

This is an NP-hard problem, which means there is no fast exact algorithm that scales to large numbers of items. That is exactly the kind of problem GAs are good at. They do not guarantee finding the absolute best answer, but they usually find very good answers quickly.

1.1 My specific problem instance

Item	Weight	Value
item0	2	6
item1	3	10
item2	4	12
item3	5	13
item4	9	18

Capacity is 15. With 5 items there are 2^5 = 32 possible chromosomes, small enough that I can verify the answer by hand. By exhaustive checking, the best valid combination is items 0, 1, 2, 3 which gives total weight 14 (under the limit) and total value 41. I will use this number 41 as the ground truth to check whether my GA actually works.

2. How a Genetic Algorithm works

A GA copies how nature does evolution. We have a population of possible solutions, called chromosomes. Each chromosome is a list of bits. For our problem, every chromosome has 5 bits, one bit for each item. Bit 1 means we include that item, bit 0 means we leave it out.

For example chromosome [1, 1, 1, 1, 0] means take items 0, 1, 2, 3 and leave item 4. Chromosome [1, 0, 1, 0, 1] means take items 0, 2, 4.

The GA repeats these steps over many generations:

2.1 Selection

We pick parent chromosomes based on how good they are. Better chromosomes get picked more often. I used **tournament selection** which works like this. Pick k random chromosomes (I used k=3), compare their fitness values, keep the best one. This biases the search toward good solutions while still keeping some random variation.

2.2 Crossover

Take two parents and mix their bits to make two children. We pick a random crossover point, then child 1 gets the first part from parent 1 and the second part from parent 2. Child 2 gets the opposite. This is how good ideas combine. If parent 1 has good genes in the front and parent 2 has good genes in the back, their child can inherit both.

Example with crossover at point 2:

```
parent1 = [1, 0, 1, 0, 1]
parent2 = [0, 1, 1, 1, 0]

child1 = [1, 0] + [1, 1, 0] = [1, 0, 1, 1, 0]
child2 = [0, 1] + [1, 0, 1] = [0, 1, 1, 0, 1]
```

2.3 Mutation

Randomly flip a bit sometimes. This adds new variations and stops the population from getting stuck. Without mutation, if a particular bit value gets lost from the population, there is no way to bring it back. Mutation keeps the search alive.

2.4 Elitism

Always keep the best chromosome from the current generation. This way we never lose progress. Without elitism, mutation could destroy the best solution we found.

2.5 The fitness function

Fitness tells the GA how good a chromosome is. For knapsack, fitness = total value, but if total weight goes over capacity, fitness = 0. This penalty makes the GA learn to avoid invalid solutions automatically. I do not have to write any explicit constraint check, the selection pressure handles it.

3. I tried Python first

Before touching MeTTa I wrote the GA in Python. Python was easier because it has random numbers built in, and lists are simple. I used the Python version to make sure I actually understood the algorithm, then planned to translate it to MeTTa.

3.1 Setting up the problem in Python

I represented the items as a list of dictionaries:

```
Items = [
    {"name": "item0", "weight": 2, "value": 6},
    {"name": "item1", "weight": 3, "value": 10},
    {"name": "item2", "weight": 4, "value": 12},
    {"name": "item3", "weight": 5, "value": 13},
    {"name": "item4", "weight": 9, "value": 18},
]
Capacity = 15
Num_items = len(Items)
```

A chromosome is just a list of 5 integers, each 0 or 1. So [1, 0, 1, 0, 1] is a valid chromosome representing the choice to take items 0, 2, and 4.

3.2 Computing weight, value, and fitness

```
def total_weight(chromosome):
    return sum(c * item["weight"] for c, item in zip(chromosome, Items))

def total_value(chromosome):
    return sum(c * item["value"] for c, item in zip(chromosome, Items))

def fitness(chromosome):
    if total_weight(chromosome) > Capacity:
        return 0
    return total_value(chromosome)
```

These are simple list comprehensions. The c * item['weight'] trick uses the fact that bit 0 contributes nothing and bit 1 contributes the full weight, which is cleaner than an if statement. The fitness function checks the constraint and returns 0 for invalid solutions.

3.3 Generating the initial random population

```
def random_chromosome():
    return [random.randint(0, 1) for _ in range(Num_items)]

def initial_population(size):
    return [random_chromosome() for _ in range(size)]
```

Each chromosome is generated by flipping 5 fair coins. The starting population is just a list of these random chromosomes. With population size 6 and 32 possible chromosomes, the starting population covers less than 20% of the search space, so the GA actually has work to do.

3.4 Tournament selection

```
def tournament_selection(population, k=3):
```

```
    contestants = random.sample(population, k)
    return max(contestants, key=fitness)
```

Two lines. Pick k random chromosomes, return the one with the highest fitness. Higher k means stronger selection pressure (the best chromosomes dominate). Lower k means more random variation. k=3 is a common default that balances both.

3.5 Single-point crossover

```
def crossover(parent1, parent2):
    if random.random() > Crossover_rate:
        return parent1[:], parent2[:]
    point = random.randint(1, Num_items - 1)
    child1 = parent1[:point] + parent2[point:]
    child2 = parent2[:point] + parent1[point:]
    return child1, child2
```

First the function decides whether to crossover at all. With Crossover_rate = 0.8, eighty percent of the time we cross, and twenty percent of the time the parents pass through unchanged. This preserves diversity. Then it picks a random point between 1 and 4 (point 0 or 5 would mean no crossover happened) and slices the lists.

3.6 Mutation

```
def mutate(chromosome):
    return [
        (1 - bit) if random.random() < Mutation_rate else bit
        for bit in chromosome
    ]
```

Each bit is checked independently. With Mutation_rate = 0.10, every bit has a 10% chance of flipping. The expression (1 - bit) flips a 0 to 1 and a 1 to 0 in one line. Across 5 bits, the expected number of flips per chromosome is 0.5, which is gentle but enough to introduce variation.

3.7 Building the next generation

```
def evolve(population):
    sorted_pop = sorted(population, key=fitness, reverse=True)
    new_population = sorted_pop[:Elitism_count]

    while len(new_population) < len(population):
        parent1 = tournament_selection(population)
        parent2 = tournament_selection(population)
        child1, child2 = crossover(parent1, parent2)
        child1 = mutate(child1)
        child2 = mutate(child2)
        new_population.append(child1)
        if len(new_population) < len(population):
            new_population.append(child2)

    return new_population
```

First, elitism. Sort the current population by fitness and copy the top one straight to the new population. Then fill the remaining slots by repeatedly: select two parents, cross them, mutate the children, and add to the new population. Keep going until the new population is the same size as the old one.

3.8 The main loop

```
population = initial_population(Population_size)
best_ever = max(population, key=fitness)
best_history = [fitness(best_ever)]

for gen in range(1, Num_generations + 1):
    population = evolve(population)
    gen_best = max(population, key=fitness)
    if fitness(gen_best) > fitness(best_ever):
        best_ever = gen_best[:]
    best_history.append(fitness(best_ever))
```

Initialize, then loop. Each iteration we evolve the population by one generation and check if the new best chromosome is better than the best we have ever seen. The best_history list lets me draw the convergence curve at the end.

3.9 Hyperparameters I used

Setting	Value	What it does
Population size	6	How many chromosomes per generation
Generations	30	How long to evolve
Tournament size	3	How many candidates fight in selection
Crossover rate	0.8	Probability that two parents will cross
Mutation rate	0.10	Probability each bit flips
Elitism count	1	Top chromosome always survives
Random seed	7	For reproducible runs

I picked these by trying different combinations. Smaller populations with longer runs make the GA work harder, which produces a more interesting convergence curve to watch. Seed 7 happened to give a particularly weak starting population, which is the most informative to study.

3.10 Python output

```
# Initial Population (seed 7)
  chrom0: [1, 0, 1, 0, 0]  fit= 18  w=  6  v= 18  items=[0, 2]
  chrom1: [0, 1, 0, 0, 0]  fit= 10  w=  3  v= 10  items=[1]
  chrom2: [0, 1, 1, 0, 0]  fit= 22  w=  7  v= 22  items=[1, 2]
  chrom3: [0, 1, 0, 0, 0]  fit= 10  w=  3  v= 10  items=[1]
  chrom4: [0, 1, 0, 0, 0]  fit= 10  w=  3  v= 10  items=[1]
  chrom5: [0, 1, 1, 0, 0]  fit= 22  w=  7  v= 22  items=[1, 2]

# Generation 1
  Best: 35  Avg: 24.0
  Best chrom: [0, 1, 1, 1, 0]  items=[1, 2, 3]

# Generation 5
  Best: 41  Avg: 31.3   <-- optimum found
  Best chrom: [1, 1, 1, 1, 0]  items=[0, 1, 2, 3]

# Generation 30
  Best: 41  Avg: 38.8

# Best Fitness Over Time
  22 -> 35 -> 41 -> 41 -> 41 -> ... -> 41

# Final Best Solution
  Best: [1, 1, 1, 1, 0]  fit= 41  w= 14  v= 41
  Selected items: [item0, item1, item2, item3]
  Optimal (41)? True
```

This is what a real GA run looks like. The starting population was weak (best was only 22, three duplicates of fitness 10). Within 2 generations the GA built up to fitness 41 by combining good partial solutions through crossover. Elitism then preserved 41 for the rest of the run.

Notice the average fitness fluctuates over time (24.0 -> 31.3 -> 38.8) even though the best stays at 41. This is the population continuing to explore new variations through crossover and mutation while elitism guards the optimum. This is exactly the exploration vs exploitation balance that the Whitley paper discusses in section 4.1.

4. Then I tried MeTTa

After Python worked, I tried to do the exact same thing in MeTTa. I thought it would be a straight translation. It was not.

4.1 The plan: copy each Python piece into MeTTa

My initial plan was to translate one Python function at a time:

Python	MeTTa equivalent I tried
list of bits	(Cons 1 (Cons 0 (Cons 1 (Cons 0 (Cons 1 Nil)))))
sum(...) total_weight	(tw $c 0) recursive walk
fitness(c) with if check	(fit $c) using if expression

parent1[:k] + parent2[k:]	(cat (tk $k $p1) (dr $k $p2))
random.randint(0, 1)	(random-int &rng 0 2)
chrom[pos] = 1 - chrom[pos]	recursive walk with index counter
max(pop, key=fitness)	(best4 $pop) using tournaments
for gen in range(N)	(next-gen4 $pop) chained calls

Some of these worked fine. Others caused MeTTa to either explode in output size or fail entirely. I will go through each problem in detail in the next section.

5. The problems I hit in MeTTa

MeTTa is not like Python. It is a symbolic, rewrite-based language where multiple matching rules can fire at once. Several Python patterns just do not work the same way. Here are the specific problems I ran into and how I solved each one.

5.1 Problem: nondeterminism explosion in list operations

I started by writing the standard recursive functions for take, drop, and concat that I needed for crossover. This is what I wrote first:

```
; Take first k elements
(= (tk 0 $l) Nil)
(= (tk $k Nil) Nil)
(= (tk $k (Cons $h $t)) (Cons $h (tk (- $k 1) $t)))

; Drop first k elements
(= (dr 0 $l) $l)
(= (dr $k Nil) Nil)
(= (dr $k (Cons $h $t)) (dr (- $k 1) $t))

; Concatenate two lists
(= (cat Nil $b) $b)
(= (cat (Cons $h $t) $b) (Cons $h (cat $t $b)))
```

These look fine. They are how you would write these functions in Haskell or any functional language. But when I called them through crossover, the output exploded into thousands of nested if-then-else expressions. The optimal answer 41 was buried somewhere inside, but the output had hundreds of partial expressions printed alongside it.

Why this happened. When MeTTa sees (tk $k $list), it tries to match against every rule that could possibly fit. For a generic list, multiple rules can match the same expression depending on what k and list evaluate to. MeTTa pursues all matching paths in parallel and returns every possible result. Each result then feeds into the next function call, multiplying the branches. The output grows exponentially.

My fix. Instead of generic recursive list operations, I wrote crossover using full pattern matching on the 5-bit chromosome structure. This destructures both parents at once and rebuilds the child directly with no intermediate function calls.

```
; Crossover at point 2 with full pattern matching
```

```
(= (cross2-a
      (Cons $a (Cons $b (Cons $c (Cons $d (Cons $e Nil)))))
      (Cons $f (Cons $g (Cons $h (Cons $i (Cons $j Nil))))))
   (Cons $a (Cons $b (Cons $h (Cons $i (Cons $j Nil))))))

; Crossover at point 3 with full pattern matching
(= (cross3-b
      (Cons $a (Cons $b (Cons $c (Cons $d (Cons $e Nil)))))
      (Cons $f (Cons $g (Cons $h (Cons $i (Cons $j Nil))))))
   (Cons $f (Cons $g (Cons $h (Cons $d (Cons $e Nil))))))
```

This works because there is exactly one matching rule per function. MeTTa cannot branch. The cost is that this only works for fixed-size chromosomes (5 bits in my case). If I wanted longer chromosomes I would need to write more pattern functions, but for this task it is fine.

5.2 Problem: no working random number generator

Python has random.randint(0, 1) for free. MeTTa has random-int in the standard library, but in my version (hyperon 0.2.10 on Windows) it does not evaluate. The expression (random-int &rng; 0 2) just stays as a symbolic atom forever. Calling it inside a fitness or crossover function then propagates the unevaluated atom into giant nested expressions.

```
!(println! (random-int &rng 0 2))
; Expected: a number 0 or 1
; Actual: just prints (random-int &rng 0 2) verbatim

; This means I cannot generate a random initial population,
; or pick random crossover points, or do random mutation
; without a working random function.
```

My fix. I made the MeTTa version deterministic. The chromosomes are 4 hand-picked starting solutions. The crossover points are fixed at 2 and 3. The mutation positions are specified explicitly. The Python version handles all the randomness in this submission. The MeTTa version still demonstrates all the core GA operations (selection, crossover, mutation, elitism) but in a reproducible way.

5.3 Problem: nested let chains kept breaking

When I tried to chain many computations together with nested let bindings, MeTTa would either give parenthesis errors or recompute the same subexpression many times. Even moving one bracket would cascade into compile errors. With 30 nested lets I lost track of the parenthesis count completely.

My fix. I avoided let chains entirely. Every chromosome and every population is defined as a top-level (= (name) value) function. Each ! statement is independent. This keeps every evaluation simple and predictable. It also makes the code easier to read because each function has a clear name.

```
; Instead of nested lets, I use named top-level functions:
(= (c1) (Cons 1 (Cons 0 (Cons 1 (Cons 0 (Cons 1 Nil))))))
(= (pop0) (Cons (c1) (Cons (c2) (Cons (c3) (Cons (c4) Nil)))))
(= (pop1) (next-gen4 (pop0)))
(= (pop2) (next-gen4 (pop1)))

; Each !(println! ...) is its own statement.
; No nesting, no parenthesis counting.
```

5.4 Problem: zero-argument functions re-evaluating

Even after fixing the explosions in crossover, I noticed something subtle. Functions like (= (c1) ...) get re-evaluated every single time they are called. So if best4 is called and it internally calls tournament four times, and each tournament calls fit twice, and fit calls (c1) inside it, that is many redundant evaluations of the same constant chromosome.

My fix. Keep the zero-argument functions as constants only (no inner function calls), and let MeTTa naturally recompute them. This is cheap because the body is just a constant Cons expression with no recursion. The bug only happens when a zero-argument function calls another zero-argument function that returns an expression that needs reduction. By making my chromosomes pure concrete data, the re-evaluation cost is negligible.

6. My MeTTa implementation in detail

Here is what my MeTTa GA looks like, going through it the same way I went through the Python code earlier.

6.1 Item data and capacity

```
(= (item-weight 0) 2)
(= (item-weight 1) 3)
(= (item-weight 2) 4)
(= (item-weight 3) 5)
(= (item-weight 4) 9)

(= (item-value 0) 6)
(= (item-value 1) 10)
(= (item-value 2) 12)
(= (item-value 3) 13)
(= (item-value 4) 18)

(= (cap) 15)
```

Items are stored as facts indexed by item number. (item-weight 2) evaluates to 4. This is a clean way to encode lookup tables in MeTTa. I chose 0-indexed to match the chromosome bit positions.

6.2 Total weight and total value

```
(= (tw Nil $i) 0)
(= (tw (Cons $b $t) $i)
   (+ (* $b (item-weight $i)) (tw $t (+ $i 1))))

(= (tv Nil $i) 0)
(= (tv (Cons $b $t) $i)
   (+ (* $b (item-value $i)) (tv $t (+ $i 1))))
```

These are recursive walks over the chromosome list. The index $i tracks which item we are currently at. The base case is empty list returns 0. The recursive case adds (bit times item-weight) and recurses on the tail with i+1. This is the MeTTa equivalent of the Python sum() comprehension.

6.3 Fitness

```
(= (fit $c)
   (if (> (tw $c 0) (cap)) 0 (tv $c 0)))
```

Identical logic to Python. If weight exceeds capacity, return 0. Otherwise return total value. The MeTTa if expression takes three arguments: condition, then-branch, else-branch.

6.4 Mutation

```
(= (fl 0) 1)
(= (fl 1) 0)

(= (mut Nil $pos $cur) Nil)
(= (mut (Cons $h $t) $pos $cur)
   (if (== $cur $pos)
       (Cons (fl $h) $t)
       (Cons $h (mut $t $pos (+ $cur 1)))))
```

```
(= (mutate $c $pos) (mut $c $pos 0))
```

fl is the bit flip helper, just two rules. mut walks the list with a position counter $cur. If the current index matches the target position, flip that bit and return the rest of the list unchanged. Otherwise, keep the current bit and recurse on the tail. The mutate wrapper just starts the walk at index 0.

6.5 Crossover (the trick that made everything work)

```
(= (cross2-a
      (Cons $a (Cons $b (Cons $c (Cons $d (Cons $e Nil)))))
      (Cons $f (Cons $g (Cons $h (Cons $i (Cons $j Nil))))))
   (Cons $a (Cons $b (Cons $h (Cons $i (Cons $j Nil))))))

(= (cross2-b
      (Cons $a (Cons $b (Cons $c (Cons $d (Cons $e Nil)))))
      (Cons $f (Cons $g (Cons $h (Cons $i (Cons $j Nil))))))
   (Cons $f (Cons $g (Cons $c (Cons $d (Cons $e Nil))))))

(= (cross3-a
      (Cons $a (Cons $b (Cons $c (Cons $d (Cons $e Nil)))))
      (Cons $f (Cons $g (Cons $h (Cons $i (Cons $j Nil))))))
   (Cons $a (Cons $b (Cons $c (Cons $i (Cons $j Nil))))))

(= (cross3-b
      (Cons $a (Cons $b (Cons $c (Cons $d (Cons $e Nil)))))
      (Cons $f (Cons $g (Cons $h (Cons $i (Cons $j Nil))))))
   (Cons $f (Cons $g (Cons $h (Cons $d (Cons $e Nil))))))
```

Each crossover function takes two parents pattern-matched as 5-element lists. The variables $a through $j represent the 10 individual bits. The body just constructs the child by picking which variable goes in which position. This is a direct, single-rule pattern match with zero branching.

6.6 Tournament and best4

```
(= (tournament $c1 $c2)
   (if (>= (fit $c1) (fit $c2)) $c1 $c2))

(= (best4 (Cons $a (Cons $b (Cons $c (Cons $d Nil)))))
   (tournament (tournament $a $b)
               (tournament $c $d)))
```

tournament compares two chromosomes and returns the fitter one. best4 finds the fittest in a population of 4 by running a single elimination bracket: a vs b in the upper bracket, c vs d in the lower bracket, and the two winners face off in the final.

6.7 Initial population

```
(= (c1) (Cons 1 (Cons 0 (Cons 1 (Cons 0 (Cons 1 Nil))))))
(= (c2) (Cons 0 (Cons 1 (Cons 1 (Cons 1 (Cons 0 Nil))))))
(= (c3) (Cons 1 (Cons 1 (Cons 1 (Cons 0 (Cons 0 Nil))))))
(= (c4) (Cons 1 (Cons 1 (Cons 0 (Cons 1 (Cons 0 Nil))))))

(= (pop0)
   (Cons (c1) (Cons (c2) (Cons (c3) (Cons (c4) Nil)))))
```

Four hand-picked starting chromosomes with diverse fitness values (36, 35, 28, 29). The population is a list of these four chromosomes. I picked these because they are different enough that crossover will produce interesting offspring.

6.8 Building the next generation with elitism

```
(= (next-gen4
      (Cons $a (Cons $b (Cons $c (Cons $d Nil)))))
   (Cons (best4 (Cons $a (Cons $b (Cons $c (Cons $d Nil)))))
   (Cons (cross2-a $a $b)
   (Cons (cross2-b $a $b)
   (Cons (cross3-b $d $c) Nil)))))
```

Slot 1 is the elite, the best chromosome from the current population. Slot 2 is cross2-a applied to the first two parents. Slot 3 is cross2-b applied to the same pair. Slot 4 is cross3-b applied to the last two parents (in reverse order to mix things up). The result is a new 4-chromosome population.

This is the MeTTa equivalent of the Python evolve function. The big structural difference is that there is no while loop. Everything is constructed by a single pattern match expression.

6.9 Generation chain

```
(= (pop1) (next-gen4 (pop0)))
(= (pop2) (next-gen4 (pop1)))
```

Each population is built from the previous one by applying next-gen4. I limited it to 2 generations because that is enough to demonstrate the GA finding the optimum. Chaining more generations is just adding more lines, but each one increases the size of the expression tree MeTTa has to evaluate.

7. MeTTa output

```
(# Initial Population)
(c1 (Cons 1 (Cons 0 (Cons 1 (Cons 0 (Cons 1 Nil))))) fit= 36 w= 15 v= 36)
(c2 (Cons 0 (Cons 1 (Cons 1 (Cons 1 (Cons 0 Nil))))) fit= 35 w= 12 v= 35)
(c3 (Cons 1 (Cons 1 (Cons 1 (Cons 0 (Cons 0 Nil))))) fit= 28 w=  9 v= 28)
(c4 (Cons 1 (Cons 1 (Cons 0 (Cons 1 (Cons 0 Nil))))) fit= 29 w= 10 v= 29)

(# Test Tournament)
(winner-c1-vs-c2 fit= 36)   <-- c1 wins (36 > 35)
(winner-c3-vs-c4 fit= 29)   <-- c4 wins (29 > 28)

(# Test Crossover)
(cross2-a-c1-c2 fit= 31 w= 11 v= 31)
(cross2-b-c1-c2 fit= 0  w= 16 v= 40)   <-- over capacity
(cross3-a-c4-c3 fit= 16 w=  5 v= 16)
(cross3-b-c4-c3 fit= 41 w= 14 v= 41)   <-- optimum found here

(# Test Mutation)
(mutate-c1-pos0 fit= 30 w= 13 v= 30)
(mutate-c1-pos3 fit= 0  w= 20 v= 49)   <-- over capacity

(# Best Pop0)
(best-pop0-fitness 36)

(# Pop1)
(best-pop1-fitness 41)   <-- optimum reached
(best-pop1-weight 14)
(best-pop1-value 41)

(# Pop2)
(best-pop2-fitness 41)   <-- elitism preserved it

(# Final Best Solution)
(best-chromosome (Cons 1 (Cons 1 (Cons 1 (Cons 1 (Cons 0 Nil))))))
(best-fitness 41)
(best-weight 14)
(best-value 41)
(items-selected item0=1 item1=1 item2=1 item3=1 item4=0)
```

The MeTTa version reaches the optimal answer (fitness 41, items 0+1+2+3) in just one generation, because cross3-b applied to c4 and c3 happens to produce the optimal chromosome [1, 1, 1, 1, 0] directly.

Pop2 keeps fitness 41 thanks to elitism. Even if all the new offspring in Pop2 were worse, the best chromosome from Pop1 would be preserved.

8. Both results side by side

	Python	MeTTa
Approach	Stochastic	Deterministic
Initial population	6 random chromosomes	4 fixed chromosomes
Best in initial pop	22	36

Generations to optimum	2	1
Crossover points	Random each time	Fixed at 2 and 3
Mutation	Per-bit, 10% probability	Fixed positions for testing
Final fitness	41	41
Final chromosome	[1, 1, 1, 1, 0]	[1, 1, 1, 1, 0]
Items selected	0, 1, 2, 3	0, 1, 2, 3
Total weight / value	14 / 41	14 / 41
Optimal answer found?	Yes	Yes

Both implementations find the same optimal solution. The Python version is a real stochastic GA with all the bells and whistles. The MeTTa version demonstrates the same algorithmic ideas but without randomness due to the random-int issue.

The reason Python takes 2 generations and MeTTa takes 1 is just luck of starting conditions. Python's random init was very weak (best fitness 22), so it had to climb. MeTTa's hand-picked init started higher (best 36), and the very first crossover I tested (cross3-b on c4 and c3) happened to produce the optimum directly. Different starting points, same destination.

9. What I learned

1. GAs are surprisingly simple to implement once you understand the four operators (selection, crossover, mutation, elitism). Python made this very easy. The whole algorithm is maybe 50 lines of real code.

2. MeTTa is fundamentally different from imperative languages. Its nondeterministic pattern-matching semantics make some normal coding patterns impossible. You have to think in terms of unique-rule pattern matches, not procedural steps.

3. Pattern matching on full data structures is more reliable in MeTTa than generic recursive functions when you want deterministic behavior. Writing four separate cross2-a, cross2-b, cross3-a, cross3-b functions felt redundant compared to a generic crossover, but it actually worked.

4. Penalty-based fitness (return 0 for invalid solutions) is a clean way to handle constraints in a GA. The selection pressure does the work of filtering out bad solutions automatically. I did not need to write any explicit constraint check or repair operator.

5. Elitism is essential. Without it, mutation can sometimes destroy a good solution. With it, fitness only ever goes up across generations, which is a nice invariant to verify.

6. Even a small population can find the optimum with enough generations, because crossover is good at combining partial solutions. This matches the schema theorem from the Whitley paper section 4. Good schemas (partial solutions like 'item 1 plus item 2') get propagated through the population proportionally to their fitness.

7. Reading the original paper (Whitley) and watching a tutorial together was much better than just one or the other. The tutorial gave me intuition, the paper gave me the math and the formal vocabulary like schema theorem, hyperplane sampling, and premature convergence.

8. Writing the same algorithm in two different languages exposed a lot about both languages. Python hides a lot of complexity. MeTTa exposes it. Both views were useful.

10. Files in this submission

`knapsack_ga.py` - the Python GA (stochastic, full version)
`knapsack.metta` - the MeTTa GA (deterministic, pattern-matched)
`Knapsack_GA_Report.pdf` - this report